

Modules

Option Explicit

```
'  
=====
```

'''

```
' Module: modAttendance  
' Purpose: Business logic and Data Access for tracking Student Attendance.  
' Author: Senior Enterprise Architect  
'  
=====
```

'''

Public Sub SetupAttendanceTable()

```
' -----  
' PURPOSE: Bootstraps the tblAttendance database structure if it does not exist.  
' -----  
  
Dim ws As Worksheet  
Dim lo As ListObject  
  
On Error GoTo ErrorHandler  
  
Set ws = ThisWorkbook.Sheets("Attendance")  
  
' Check if table already exists  
On Error Resume Next
```

```
Set lo = ws.ListObjects("tblAttendance")
```

```
On Error GoTo ErrorHandler
```

```
If lo Is Nothing Then
```

```
    ' Build Headers
```

```
    ws.Cells(1, 1).Value = "AttendanceID"
```

```
    ws.Cells(1, 2).Value = "ClassDate"
```

```
    ws.Cells(1, 3).Value = "StudentID"
```

```
    ws.Cells(1, 4).Value = "Status"    ' e.g., Present, Absent, Late
```

```
    ws.Cells(1, 5).Value = "RecordedBy" ' Audit trail
```

```
    ' Create ListObject
```

```
    Set lo = ws.ListObjects.Add(xlSrcRange, ws.Range("A1:E2"), , xlYes)
```

```
    lo.Name = "tblAttendance"
```

```
    lo.TableStyle = "TableStyleMedium3" ' Green styling for distinction
```

```
    Debug.Print "System architecture updated: tblAttendance created."
```

```
End If
```

```
Exit Sub
```

```
ErrorHandler:
```

```
    MsgBox "Error creating tblAttendance: " & Err.Description, vbCritical, "Database Setup"
```

```
End Sub
```

```
Public Function RecordAttendance(ByVal classDate As Date, ByVal studentID As Long, ByVal Status As  
String) As Boolean
```

```

' -----
' PURPOSE: Safely writes a single attendance record to the database.
' -----

Dim lo As ListObject

Dim lr As ListRow

Dim newID As Long

On Error GoTo ErrorHandler

RecordAttendance = False

' 1. Validate inputs

If studentID <= 0 Or Trim(Status) = "" Then

    Err.Raise vbObjectError + 1, , "Invalid StudentID or Status."

End If

' 2. Connect to Database

Set lo = modDatabase.GetTable("tblAttendance")

If lo Is Nothing Then Err.Raise vbObjectError + 2, , "tblAttendance not found. Run
SetupAttendanceTable first."

' 3. Generate sequential ID

newID = modDatabase.GetNextID("tblAttendance", "AttendanceID")

' 4. Write data securely using Intersect to avoid column-shift bugs

Set lr = lo.ListRows.Add

Intersect(lr.Range, lo.ListColumns("AttendanceID").Range).Value = newID

```

```
Intersect(lr.Range, lo.ListColumns("ClassDate").Range).Value = classDate
```

```
Intersect(lr.Range, lo.ListColumns("StudentID").Range).Value = studentID
```

```
Intersect(lr.Range, lo.ListColumns("Status").Range).Value = Trim(Status)
```

```
' Capture Audit Trail (If logged in, use their name. Otherwise, system default)
```

```
If g_IsLoggedIn Then
```

```
    Intersect(lr.Range, lo.ListColumns("RecordedBy").Range).Value = g_CurrentUsername
```

```
Else
```

```
    Intersect(lr.Range, lo.ListColumns("RecordedBy").Range).Value = "SYSTEM"
```

```
End If
```

```
RecordAttendance = True
```

```
Exit Function
```

```
ErrorHandler:
```

```
MsgBox "Database Error: Could not save attendance." & vbCrLf & _
```

```
    "Details: " & Err.Description, vbCritical, "modAttendance.RecordAttendance"
```

```
End Function
```

```
Option Explicit
```

```
,
```

```
=====
```

```
===
```

' Module: modDatabase

' Purpose: Data Access Layer (DAL). Centralized engine for reading/writing ListObjects.

' Author: Senior Enterprise Architect

'

=====
===

Public Function GetTable(ByVal TableName As String) As ListObject

' -----

' PURPOSE: Safely retrieves a ListObject by its name from any worksheet.

' RETURNS: ListObject or Nothing if not found.

' -----

Dim ws As Worksheet

Dim lo As ListObject

On Error GoTo ErrorHandler

' Loop through all sheets. Once found, return it and exit.

For Each ws In ThisWorkbook.Worksheets

On Error Resume Next ' Temporarily ignore errors if sheet has no tables

Set lo = ws.ListObjects(TableName)

On Error GoTo ErrorHandler ' Restore normal error handling

If Not lo Is Nothing Then

Set GetTable = lo

Exit Function

End If

Next ws

' If we get here, the table doesn't exist.

Set GetTable = Nothing

Exit Function

ErrorHandler:

MsgBox "Database Error: Could not retrieve table '" & TableName & "'." & vbCrLf & _

"Details: " & Err.Description, vbCritical, "modDatabase.GetTable"

Set GetTable = Nothing

End Function

Public Function GetNextID(ByVal TableName As String, ByVal ColumnName As String) As Long

' -----

' PURPOSE: Generates an automatic, sequential, unique ID for database normalization.

' RETURNS: The next available Long integer.

' -----

Dim lo As ListObject

Dim colIndex As Integer

Dim MaxID As Double

On Error GoTo ErrorHandler

Set lo = GetTable(TableName)

```
If lo Is Nothing Then Err.Raise vbObjectError + 1, , "Table missing."
```

```
' Handle empty table scenario
```

```
If lo.ListRows.count = 0 Or lo.DataBodyRange Is Nothing Then
```

```
    GetNextID = 1
```

```
    Exit Function
```

```
End If
```

```
' Find column index and calculate Max using Excel's built-in fast function
```

```
colIndex = lo.ListColumns(ColumnName).Index
```

```
MaxID = Application.WorksheetFunction.Max(lo.ListColumns(colIndex).DataBodyRange)
```

```
GetNextID = CLng(MaxID) + 1
```

```
Exit Function
```

```
ErrorHandler:
```

```
MsgBox "Database Error: Failed to generate ID for '" & TableName & "'." & vbCrLf & _
```

```
    "Details: " & Err.Description, vbCritical, "modDatabase.GetNextID"
```

```
GetNextID = -1 ' Indicates failure
```

```
End Function
```

```
Public Function FindRecordRow(ByVal TableName As String, ByVal ColumnName As String, ByVal  
SearchValue As Variant) As Long
```

```
    ' -----
```

```
    ' PURPOSE: Searches a table for a value and returns the ListRow index (1-based).
```

```
    ' RETURNS: Row index if found, 0 if not found.
```

```

' -----

Dim lo As ListObject

Dim rngCol As Range

Dim foundCell As Range


On Error GoTo ErrorHandler


Set lo = GetTable(TableName)

If lo Is Nothing Then Exit Function

If lo.DataBodyRange Is Nothing Then Exit Function ' Table has headers but no data


Set rngCol = lo.ListColumns(ColumnName).DataBodyRange


' LookAt:=xlWhole ensures we match exact strings (e.g., "admin" doesn't match "admin2")

Set foundCell = rngCol.Find(What:=SearchValue, LookIn:=xlValues, LookAt:=xlWhole,
MatchCase:=False)


If Not foundCell Is Nothing Then

    ' Calculate the row relative to the DataBodyRange

    FindRecordRow = foundCell.Row - lo.HeaderRowRange.Row

Else

    FindRecordRow = 0 ' Not found

End If


Exit Function

```


ErrorHandler:

```
MsgBox "Database Error: Search failed in '" & TableName & "'." & vbCrLf & _
```

```
"Details: " & Err.Description, vbCritical, "modDatabase.FindRecordRow"
```

```
FindRecordRow = -1
```

End Function

Option Explicit

```
'
```

```
=====
```

```
===
```

```
' Module: modGlobal
```

```
' Purpose: Stores application-wide constants, enumerations, and global variables.
```

```
' Author: Senior Enterprise Architect
```

```
'
```

```
=====
```

```
===
```

```
' --- SYSTEM CONSTANTS ---
```

```
' Constants never change during runtime. They save memory and prevent typos.
```

```
Public Const SYS_NAME As String = "Sunday School Management System"
```

```
Public Const SYS_VERSION As String = "1.0.0"
```

```
Public Const SYS_AUTHOR As String = "System Admin"
```

```
' --- ENUMERATIONS ---
```

```
' Enums map integers to readable names.
```

' Later, we can write: If g_CurrentUserRole = UserRole.Admin Then...

Public Enum UserRole

Admin = 1

Teacher = 2

Viewer = 3

End Enum

Public Enum RecordStatus

Active = 1

Inactive = 0

Deleted = -1

End Enum

' --- GLOBAL VARIABLES ---

' Prefix with 'g_' to instantly identify global scope anywhere in the code.

Public g_CurrentUserID As Long

Public g_CurrentUsername As String

Public g_CurrentUserRole As UserRole

Public g_IsLoggedIn As Boolean

Option Explicit

,

=====

```

' Module: modLogs

' Purpose: Enterprise audit trail. Tracks user actions across the system.

' Author: Senior Enterprise Architect
'
=====

Public Sub WriteLog(ByVal ActionType As String, ByVal ModuleName As String, ByVal Details As String)

    ' -----

    ' PURPOSE: Safely writes a log entry to tblLogs in the background.

    ' ARCHITECTURE: This fails silently. If logging breaks (e.g., table locked),
    '               it will NOT crash the user's primary task (like saving a student).
    ' -----

    Dim lo As ListObject

    Dim lr As ListRow

    Dim newID As Long

    On Error GoTo ErrorHandler

    ' 1. Connect to Database (tblLogs was created by modSystemSetup)

    Set lo = modDatabase.GetTable("tblLogs")

    If lo Is Nothing Then Exit Sub

    ' 2. Generate sequential ID

    newID = modDatabase.GetNextID("tblLogs", "LogID")

    ' 3. Add Row

```

```
Set lr = lo.ListRows.Add
```

```
' 4. Write Data Securely (Using Intersect prevents column-shift bugs)
```

```
Intersect(lr.Range, lo.ListColumns("LogID").Range).Value = newID
```

```
Intersect(lr.Range, lo.ListColumns("Timestamp").Range).Value = Now
```

```
' Record UserID if logged in, otherwise record 0 (System)
```

```
If g_IsLoggedIn Then
```

```
    Intersect(lr.Range, lo.ListColumns("UserID").Range).Value = g_CurrentUserID
```

```
Else
```

```
    Intersect(lr.Range, lo.ListColumns("UserID").Range).Value = 0
```

```
End If
```

```
' Truncate strings to prevent database overflow errors
```

```
Intersect(lr.Range, lo.ListColumns("Action").Range).Value = Left(ActionType, 50)
```

```
Intersect(lr.Range, lo.ListColumns("Module").Range).Value = Left(ModuleName, 50)
```

```
Intersect(lr.Range, lo.ListColumns("Details").Range).Value = Left(Details, 255)
```

```
Exit Sub
```

```
ErrorHandler:
```

```
' DO NOT use MsgBox here. An enterprise system logs silently.
```

```
Debug.Print "SYSTEM ERROR: Logging Failed - " & Err.Description
```

```
End Sub
```

Option Explicit

'

=====

' Module: modLogs

' Purpose: Enterprise audit trail. Tracks user actions across the system.

' Author: Senior Enterprise Architect

'

=====

Public Sub WriteLog(ByVal ActionType As String, ByVal ModuleName As String, ByVal Details As String)

' -----

' PURPOSE: Safely writes a log entry to tblLogs in the background.

' ARCHITECTURE: This fails silently. If logging breaks (e.g., table locked),

' it will NOT crash the user's primary task (like saving a student).

' -----

Dim lo As ListObject

Dim lr As ListRow

Dim newID As Long

On Error GoTo ErrorHandler

' 1. Connect to Database (tblLogs was created by modSystemSetup)

Set lo = modDatabase.GetTable("tblLogs")

If lo Is Nothing Then Exit Sub

' 2. Generate sequential ID

newID = modDatabase.GetNextID("tblLogs", "LogID")

' 3. Add Row

Set lr = lo.ListRows.Add

' 4. Write Data Securely (Using Intersect prevents column-shift bugs)

Intersect(lr.Range, lo.ListColumns("LogID").Range).Value = newID

Intersect(lr.Range, lo.ListColumns("Timestamp").Range).Value = Now

' Record UserID if logged in, otherwise record 0 (System)

If g_IsLoggedIn Then

Intersect(lr.Range, lo.ListColumns("UserID").Range).Value = g_CurrentUserID

Else

Intersect(lr.Range, lo.ListColumns("UserID").Range).Value = 0

End If

' Truncate strings to prevent database overflow errors

Intersect(lr.Range, lo.ListColumns("Action").Range).Value = Left(ActionType, 50)

Intersect(lr.Range, lo.ListColumns("Module").Range).Value = Left(ModuleName, 50)

Intersect(lr.Range, lo.ListColumns("Details").Range).Value = Left(Details, 255)

Exit Sub

ErrorHandler:

' DO NOT use MsgBox here. An enterprise system logs silently.

Debug.Print "SYSTEM ERROR: Logging Failed - " & Err.Description

End Sub

Option Explicit

'

=====

===

' Module: modMain

' Purpose: Primary controller for application startup, shutdown, and UI lockdown.

' Author: Senior Enterprise Architect

'

=====

===

Public Sub StartApplication()

' -----

' PURPOSE: Secures the workbook and launches the authentication gateway.

' -----

On Error GoTo ErrorHandler

' 1. Optimize load time and hide background flickering

modUtilities.OptimizeApp True

' 2. Enforce Security: Hide all sheets except the 'Temp' landing page

LockdownWorkbook

' 3. Restore application state so the UserForm renders properly

modUtilities.OptimizeApp False

' 4. Launch the Login Gateway

' vbModal ensures the user CANNOT click anywhere else in Excel until this form closes

frmLogin.Show vbModal

' Note: The code pauses here until frmLogin is unloaded.

' Once unloaded, we check if the user actually logged in.

If g_IsLoggedIn Then

 ' Launch the main dashboard switchboard

 frmDashboard.Show vbModal

 ' Once the dashboard is closed/unloaded, the system should shut down

 Call CloseApplication

Else

 ' User cancelled or failed login. Close the application.

 Call CloseApplication

End If

Exit Sub

ErrorHandler:

modUtilities.OptimizeApp False


```
MsgBox "Critical Startup Error: " & Err.Description, vbCritical, "System Error"
```

```
End Sub
```

```
Private Sub LockdownWorkbook()
```

```
    ' -----
```

```
    ' PURPOSE: Uses xlSheetVeryHidden to protect database sheets from users.
```

```
    ' -----
```

```
    Dim ws As Worksheet
```

```
    On Error GoTo ErrorHandler
```

```
    ' Ensure Temp sheet is visible first (Excel requires at least 1 visible sheet)
```

```
    ThisWorkbook.Sheets("Temp").Visible = xlSheetVisible
```

```
    ThisWorkbook.Sheets("Temp").Activate
```

```
    ' Loop through all sheets and hide everything except Temp
```

```
    For Each ws In ThisWorkbook.Worksheets
```

```
        If ws.Name <> "Temp" Then
```

```
            ws.Visible = xlSheetVeryHidden
```

```
        End If
```

```
    Next ws
```

```
Exit Sub
```

```
ErrorHandler:
```

```
    Err.Raise Err.Number, "LockdownWorkbook", Err.Description
```

End Sub

Public Sub CloseApplication()

' -----

' PURPOSE: Safely saves and shuts down the system.

' -----

On Error Resume Next

modSecurity.LogoutUser

' Save the workbook securely

ThisWorkbook.Save

' Close Excel (If this is the only workbook open, quit the app. Otherwise, just close this file)

If Application.Workbooks.count = 1 Then

Application.Quit

Else

ThisWorkbook.Close SaveChanges:=False

End If

End Sub

Option Explicit

```

'
=====
===

' Module: modRanking

' Purpose: Business logic for calculating and assigning student ranks.

' Author: Senior Enterprise Architect
'
=====
===

```

```

Public Sub CalculateClassRanks(ByVal ExamName As String, ByVal ClassLevel As String)

```

```

' -----
' PURPOSE: Calculates and updates the 'ClassRank' for all students in a specific
'         class and exam, based on their AverageScore.
' -----

```

```

Dim lo As ListObject
Dim lr As ListRow
Dim dataRange As Range
Dim i As Long, j As Long
Dim tempVal As Variant
Dim studentData() As Variant
Dim recordCount As Long
Dim currentRank As Long
Dim currentAverage As Double
Dim previousAverage As Double

```

```

On Error GoTo ErrorHandler

```

```

' 1. Connect to Database

Set lo = modDatabase.GetTable("tblResults")

If lo Is Nothing Then Err.Raise vbObjectError + 1, , "tblResults not found."


' 2. Count matching records to size our array

recordCount = 0

For Each lr In lo.ListRows

    If Intersect(lr.Range, lo.ListColumns("ExamName").Range).Value = ExamName And _
        Intersect(lr.Range, lo.ListColumns("ClassLevel").Range).Value = ClassLevel Then
        recordCount = recordCount + 1
    End If

Next lr


If recordCount = 0 Then

    MsgBox "No results found for " & ExamName & " in " & ClassLevel & ".", vbInformation, "Ranking Status"

    Exit Sub

End If


' 3. Initialize Array (Row, Col: 1=ResultID, 2=AverageScore, 3=Rank)

ReDim studentData(1 To recordCount, 1 To 3)


' 4. Populate Array

Dim arrIndex As Long

arrIndex = 1

```

```

For Each lr In lo.ListRows

    If Intersect(lr.Range, lo.ListColumns("ExamName").Range).Value = ExamName And _
        Intersect(lr.Range, lo.ListColumns("ClassLevel").Range).Value = ClassLevel Then

        studentData(arrIndex, 1) = Intersect(lr.Range, lo.ListColumns("ResultID").Range).Value

        studentData(arrIndex, 2) = Intersect(lr.Range, lo.ListColumns("AverageScore").Range).Value

        arrIndex = arrIndex + 1

    End If

Next lr

```

' 5. Sort Array by AverageScore (Descending) - Simple Bubble Sort

```

For i = 1 To recordCount - 1

    For j = i + 1 To recordCount

        If studentData(j, 2) > studentData(i, 2) Then

            ' Swap ID

            tempVal = studentData(i, 1)

            studentData(i, 1) = studentData(j, 1)

            studentData(j, 1) = tempVal

            ' Swap Average

            tempVal = studentData(i, 2)

            studentData(i, 2) = studentData(j, 2)

            studentData(j, 2) = tempVal

        End If

    Next j

Next i

```

' 6. Assign Ranks (Handling Ties)

currentRank = 1

previousAverage = -1

For i = 1 To recordCount

currentAverage = studentData(i, 2)

If currentAverage <> previousAverage And previousAverage <> -1 Then

currentRank = i ' Standard Competition Ranking: Rank jumps after a tie

End If

studentData(i, 3) = currentRank

previousAverage = currentAverage

Next i

' 7. Write Ranks back to Database

For i = 1 To recordCount

' Find the correct row using our generic database function

Dim targetRow As Long

targetRow = modDatabase.FindRecordRow("tblResults", "ResultID", studentData(i, 1))

If targetRow > 0 Then

Intersect(lo.ListRows(targetRow).Range, lo.ListColumns("ClassRank").Range).Value =
studentData(i, 3)

End If

Next i

MsgBox "Ranking complete for " & ClassLevel & " (" & ExamName & ").", vbInformation, "Ranking
Success"

Exit Sub

ErrorHandler:

MsgBox "Error calculating ranks: " & Err.Description, vbCritical, "modRanking.CalculateClassRanks"

End Sub

Option Explicit

'

=====
===

' Module: modReportCard

' Purpose: Business logic for aggregating data and generating printable report cards.

' Author: Senior Enterprise Architect

'

=====
===

Public Sub SetupReportCardTemplate()

' -----

' PURPOSE: Bootstraps a basic Report Card template on the ReportCard sheet.

' -----

Dim ws As Worksheet

On Error GoTo ErrorHandler

Set ws = ThisWorkbook.Sheets("ReportCard")

' Clear existing template

ws.Cells.Clear

' Build basic structure

ws.Range("A1:F1").Merge

ws.Range("A1").Value = SYS_NAME & " - Official Report Card"

ws.Range("A1").Font.Bold = True

ws.Range("A1").Font.Size = 14

ws.Range("A1").HorizontalAlignment = xlCenter

ws.Range("A3").Value = "Student Name:"

ws.Range("A4").Value = "Class/Section:"

ws.Range("D3").Value = "Exam Term:"

ws.Range("D4").Value = "Student ID:"

ws.Range("A6").Value = "Subject"

ws.Range("B6").Value = "Score"

ws.Range("A6:B6").Font.Bold = True


```
ws.Range("A6:B6").Borders(xlEdgeBottom).LineStyle = xlContinuous
```

```
ws.Range("A7").Value = "Faith"
```

```
ws.Range("A8").Value = "Geez"
```

```
ws.Range("A9").Value = "History"
```

```
ws.Range("A10").Value = "Morality"
```

```
ws.Range("A11").Value = "Liturgy"
```

```
ws.Range("A12").Value = "Saints"
```

```
ws.Range("D6").Value = "Summary"
```

```
ws.Range("E6").Value = "Value"
```

```
ws.Range("D6:E6").Font.Bold = True
```

```
ws.Range("D6:E6").Borders(xlEdgeBottom).LineStyle = xlContinuous
```

```
ws.Range("D7").Value = "Total Score:"
```

```
ws.Range("D8").Value = "Average:"
```

```
ws.Range("D9").Value = "Grade:"
```

```
ws.Range("D10").Value = "Class Rank:"
```

```
ws.Range("A14").Value = "Attendance Summary"
```

```
ws.Range("A14:B14").Merge
```

```
ws.Range("A14").Font.Bold = True
```

```
ws.Range("A15").Value = "Days Present:"
```

```
ws.Range("A16").Value = "Days Absent:"
```

```
ws.Columns("A:E").AutoFit
```

```
Debug.Print "System architecture updated: ReportCard template initialized."
```

```
Exit Sub
```

```
ErrorHandler:
```

```
MsgBox "Error setting up template: " & Err.Description, vbCritical, "Setup Error"
```

```
End Sub
```

```
Public Function GenerateReportCard(ByVal TargetStudentID As Long, ByVal ExamName As String) As Boolean
```

```
' -----
```

```
' PURPOSE: Populates the ReportCard template with a specific student's data.
```

```
' -----
```

```
Dim wsTemplate As Worksheet
```

```
Dim loStudents As ListObject
```

```
Dim loResults As ListObject
```

```
Dim studentRow As Long
```

```
Dim resultRow As Long
```

```
On Error GoTo ErrorHandler
```

```
GenerateReportCard = False
```

```
Set wsTemplate = ThisWorkbook.Sheets("ReportCard")
```

```
' 1. Fetch Student Data
```

```
Set loStudents = modDatabase.GetTable("tblStudents")
```

```
studentRow = modDatabase.FindRecordRow("tblStudents", "StudentID", TargetStudentID)
```

```
If studentRow = 0 Then
```

```
    MsgBox "Student ID " & TargetStudentID & " not found.", vbExclamation, "Data Error"
```

```
    Exit Function
```

```
End If
```

```
Dim sName As String, sClass As String, sSection As String
```

```
sName = Intersect(loStudents.ListRows(studentRow).Range,  
loStudents.ListColumns("FirstName").Range).Value & " " & _
```

```
    Intersect(loStudents.ListRows(studentRow).Range,  
loStudents.ListColumns("LastName").Range).Value
```

```
sClass = Intersect(loStudents.ListRows(studentRow).Range,  
loStudents.ListColumns("Class").Range).Value
```

```
sSection = Intersect(loStudents.ListRows(studentRow).Range,  
loStudents.ListColumns("Section").Range).Value
```

```
' 2. Fetch Result Data
```

```
Set loResults = modDatabase.GetTable("tblResults")
```

```
' We need a custom search because we must match both StudentID AND ExamName
```

```
Dim lr As ListRow
```

```
resultRow = 0
```

```
For Each lr In loResults.ListRows
```

```
    If Intersect(lr.Range, loResults.ListColumns("StudentID").Range).Value = TargetStudentID And _
```

```
        Intersect(lr.Range, loResults.ListColumns("ExamName").Range).Value = ExamName Then
```

```
        resultRow = lr.Index
```

Exit For

End If

Next Ir

If resultRow = 0 Then

MsgBox "No results found for " & sName & " in exam: " & ExamName, vbExclamation, "Data Error"

Exit Function

End If

' 3. Populate the Template

' Header Info

wsTemplate.Range("B3").Value = sName

wsTemplate.Range("B4").Value = sClass & " - Section " & sSection

wsTemplate.Range("E3").Value = ExamName

wsTemplate.Range("E4").Value = TargetStudentID

' Subject Scores

wsTemplate.Range("B7").Value = Intersect(loResults.ListRows(resultRow).Range,
loResults.ListColumns("Faith").Range).Value

wsTemplate.Range("B8").Value = Intersect(loResults.ListRows(resultRow).Range,
loResults.ListColumns("Geez").Range).Value

wsTemplate.Range("B9").Value = Intersect(loResults.ListRows(resultRow).Range,
loResults.ListColumns("History").Range).Value

wsTemplate.Range("B10").Value = Intersect(loResults.ListRows(resultRow).Range,
loResults.ListColumns("Morality").Range).Value

wsTemplate.Range("B11").Value = Intersect(loResults.ListRows(resultRow).Range,
loResults.ListColumns("Liturgy").Range).Value

```
wsTemplate.Range("B12").Value = Intersect(loResults.ListRows(resultRow).Range,  
loResults.ListColumns("Saints").Range).Value
```

```
' Summary
```

```
wsTemplate.Range("E7").Value = Intersect(loResults.ListRows(resultRow).Range,  
loResults.ListColumns("TotalScore").Range).Value
```

```
wsTemplate.Range("E8").Value = Intersect(loResults.ListRows(resultRow).Range,  
loResults.ListColumns("AverageScore").Range).Value
```

```
wsTemplate.Range("E9").Value = Intersect(loResults.ListRows(resultRow).Range,  
loResults.ListColumns("Grade").Range).Value
```

```
wsTemplate.Range("E10").Value = Intersect(loResults.ListRows(resultRow).Range,  
loResults.ListColumns("ClassRank").Range).Value
```

```
' 4. Calculate Attendance (Basic implementation)
```

```
Dim loAttendance As ListObject
```

```
Set loAttendance = modDatabase.GetTable("tblAttendance")
```

```
Dim daysPresent As Integer, daysAbsent As Integer
```

```
daysPresent = 0
```

```
daysAbsent = 0
```

```
If Not loAttendance Is Nothing Then
```

```
For Each lr In loAttendance.ListRows
```

```
    If Intersect(lr.Range, loAttendance.ListColumns("StudentID").Range).Value = TargetStudentID  
Then
```

```
        If Intersect(lr.Range, loAttendance.ListColumns("Status").Range).Value = "Present" Then
```

```
            daysPresent = daysPresent + 1
```

```
        ElseIf Intersect(lr.Range, loAttendance.ListColumns("Status").Range).Value = "Absent" Then
```

```
            daysAbsent = daysAbsent + 1
```

```

        End If

    End If

Next lr

End If

wsTemplate.Range("B15").Value = daysPresent

wsTemplate.Range("B16").Value = daysAbsent


' Format cells (e.g., center align values)
wsTemplate.Range("B7:B12,E7:E10,B15:B16").HorizontalAlignment = xlCenter


GenerateReportCard = True

Exit Function


ErrorHandler:

    MsgBox "Error generating report card: " & Err.Description, vbCritical,
"modReportCard.GenerateReportCard"

End Function

```

```

Option Explicit

'
=====
===

' Module: modResults

' Purpose: Business logic for Examination Results, Calculations, and Grading.

' Author: Senior Enterprise Architect

```

```
'  
=====
```

```
Public Sub SetupResultsTable()
```

```
' -----
```

```
' PURPOSE: Bootstraps the tblResults database structure if it does not exist.
```

```
' -----
```

```
Dim ws As Worksheet
```

```
Dim lo As ListObject
```

```
On Error GoTo ErrorHandler
```

```
Set ws = ThisWorkbook.Sheets("Results")
```

```
' Check if table already exists
```

```
On Error Resume Next
```

```
Set lo = ws.ListObjects("tblResults")
```

```
On Error GoTo ErrorHandler
```

```
If lo Is Nothing Then
```

```
    ' Build Headers
```

```
    ws.Cells(1, 1).Value = "ResultID"
```

```
    ws.Cells(1, 2).Value = "ExamName" ' e.g., "Term 1", "Midterm"
```

```
    ws.Cells(1, 3).Value = "ClassLevel"
```

```
    ws.Cells(1, 4).Value = "StudentID"
```

```
ws.Cells(1, 5).Value = "Faith"
ws.Cells(1, 6).Value = "Geez"
ws.Cells(1, 7).Value = "History"
ws.Cells(1, 8).Value = "Morality"
ws.Cells(1, 9).Value = "Liturgy"
ws.Cells(1, 10).Value = "Saints"
ws.Cells(1, 11).Value = "TotalScore"
ws.Cells(1, 12).Value = "AverageScore"
ws.Cells(1, 13).Value = "Grade"
ws.Cells(1, 14).Value = "ClassRank" ' Calculated later via ranking engine
ws.Cells(1, 15).Value = "RecordedBy"
```

```
' Create ListObject
```

```
Set lo = ws.ListObjects.Add(xlSrcRange, ws.Range("A1:O2"), , xlYes)
```

```
lo.Name = "tblResults"
```

```
lo.TableStyle = "TableStyleMedium5" ' Blue styling for distinction
```

```
Debug.Print "System architecture updated: tblResults created."
```

```
End If
```

```
Exit Sub
```

```
ErrorHandler:
```

```
MsgBox "Error creating tblResults: " & Err.Description, vbCritical, "Database Setup"
```

```
End Sub
```



```

Public Function SaveStudentResult(ByVal ExamName As String, ByVal ClassLevel As String, _

    ByVal studentID As Long, ByVal Faith As Double, _

    ByVal Geez As Double, ByVal History As Double, _

    ByVal Morality As Double, ByVal Liturgy As Double, _

    ByVal Saints As Double) As Boolean

' -----

' PURPOSE: Calculates total/average/grade and saves the result record.

' -----

Dim lo As ListObject

Dim lr As ListRow

Dim newID As Long

Dim Total As Double

Dim Average As Double

Dim Grade As String

On Error GoTo ErrorHandler

SaveStudentResult = False

' 1. Validate inputs

If studentID <= 0 Or Trim(ExamName) = "" Then

    Err.Raise vbObjectError + 1, , "Invalid StudentID or Exam Name."

End If

' 2. Perform Business Math (Total, Average, Percentage)

Total = Faith + Geez + History + Morality + Liturgy + Saints

```

Average = Total / 6 ' 6 Subjects

Grade = GetLetterGrade(Average)

' 3. Connect to Database

Set lo = modDatabase.GetTable("tblResults")

If lo Is Nothing Then Err.Raise vbObjectError + 2, , "tblResults not found. Run SetupResultsTable first."

' 4. Generate sequential ID

newID = modDatabase.GetNextID("tblResults", "ResultID")

' 5. Write data securely

Set lr = lo.ListRows.Add

Intersect(lr.Range, lo.ListColumns("ResultID").Range).Value = newID

Intersect(lr.Range, lo.ListColumns("ExamName").Range).Value = Trim(ExamName)

Intersect(lr.Range, lo.ListColumns("ClassLevel").Range).Value = Trim(ClassLevel)

Intersect(lr.Range, lo.ListColumns("StudentID").Range).Value = studentID

' Subjects

Intersect(lr.Range, lo.ListColumns("Faith").Range).Value = Faith

Intersect(lr.Range, lo.ListColumns("Geez").Range).Value = Geez

Intersect(lr.Range, lo.ListColumns("History").Range).Value = History

Intersect(lr.Range, lo.ListColumns("Morality").Range).Value = Morality

Intersect(lr.Range, lo.ListColumns("Liturgy").Range).Value = Liturgy

Intersect(lr.Range, lo.ListColumns("Saints").Range).Value = Saints

' Calculations

Intersect(lr.Range, lo.ListColumns("TotalScore").Range).Value = Total

Intersect(lr.Range, lo.ListColumns("AverageScore").Range).Value = Round(Average, 2)

Intersect(lr.Range, lo.ListColumns("Grade").Range).Value = Grade

' Audit Trail

If g_IsLoggedIn Then

Intersect(lr.Range, lo.ListColumns("RecordedBy").Range).Value = g_CurrentUsername

Else

Intersect(lr.Range, lo.ListColumns("RecordedBy").Range).Value = "SYSTEM"

End If

SaveStudentResult = True

Exit Function

ErrorHandler:

MsgBox "Database Error: Could not save results." & vbCrLf & _

"Details: " & Err.Description, vbCritical, "modResults.SaveStudentResult"

End Function

Private Function GetLetterGrade(ByVal AverageScore As Double) As String

' -----

' PURPOSE: Private helper to calculate Ethiopian/Standard Letter Grades.

' -----

Select Case AverageScore

```
Case Is >= 90
    GetLetterGrade = "A"
Case Is >= 80
    GetLetterGrade = "B"
Case Is >= 70
    GetLetterGrade = "C"
Case Is >= 60
    GetLetterGrade = "D"
Case Else
    GetLetterGrade = "F"
End Select
End Function
```

Option Explicit

```
'
=====
' Module: modSchedule
' Purpose: Business logic for managing the Sunday School Weekly Timetable.
' Author: Senior Enterprise Architect
'
=====
```

Public Sub SetupScheduleTable()

```
' -----
```

' PURPOSE: Bootstraps the tblSchedule database structure.

' -----

Dim ws As Worksheet

Dim lo As ListObject

Dim headers As String

On Error GoTo ErrorHandler

Set ws = ThisWorkbook.Sheets("Schedule")

headers = "ScheduleID,DayOfWeek,StartTime,EndTime,ClassLevel,Subject,TeacherID,Room"

' Check if table exists

On Error Resume Next

Set lo = ws.ListObjects("tblSchedule")

On Error GoTo ErrorHandler

If lo Is Nothing Then

Dim arrHeaders As Variant, i As Integer

arrHeaders = Split(headers, ",")

For i = LBound(arrHeaders) To UBound(arrHeaders)

ws.Cells(1, i + 1).Value = arrHeaders(i)

Next i

Set lo = ws.ListObjects.Add(xlSrcRange, ws.Range(ws.Cells(1, 1), ws.Cells(2, UBound(arrHeaders) + 1)), , xlYes)

lo.Name = "tblSchedule"

```
lo.TableStyle = "TableStyleMedium6" ' Distinct orange/gold style for Schedule
```

```
modLogs.WriteLog "INIT", "modSchedule", "Schedule table initialized."
```

```
End If
```

```
Exit Sub
```

```
ErrorHandler:
```

```
MsgBox "Error in SetupScheduleTable: " & Err.Description, vbCritical
```

```
End Sub
```

```
Public Function AddScheduleSlot( _
```

```
    ByVal DayName As String, _
```

```
    ByVal StartT As String, _
```

```
    ByVal EndT As String, _
```

```
    ByVal ClassLvl As String, _
```

```
    ByVal Subj As String, _
```

```
    ByVal TeacherID As Long, _
```

```
    ByVal RoomNo As String) As Boolean
```

```
' -----
```

```
' PURPOSE: Adds a new slot to the timetable after validating constraints.
```

```
' -----
```

```
Dim lo As ListObject
```

```
Dim lr As ListRow
```

```
Dim newID As Long
```

On Error GoTo ErrorHandler

' 1. Check for Conflicts (Simple version: Is the teacher already booked at this time/day?)

If HasConflict(DayName, StartT, TeacherID) Then

Err.Raise vbObjectError + 600, "modSchedule", "Conflict Detected: This teacher is already scheduled for another class at this time."

End If

' 2. Get Database Reference

Set lo = ThisWorkbook.Sheets("Schedule").ListObjects("tblSchedule")

newID = modDatabase.GetNextID("tblSchedule", "ScheduleID")

' 3. Add Record

Set lr = lo.ListRows.Add

With lr.Range

Intersect(.Cells, lo.ListColumns("ScheduleID").Range).Value = newID

Intersect(.Cells, lo.ListColumns("DayOfWeek").Range).Value = DayName

Intersect(.Cells, lo.ListColumns("StartTime").Range).Value = StartT

Intersect(.Cells, lo.ListColumns("EndTime").Range).Value = EndT

Intersect(.Cells, lo.ListColumns("ClassLevel").Range).Value = ClassLvl

Intersect(.Cells, lo.ListColumns("Subject").Range).Value = Subj

Intersect(.Cells, lo.ListColumns("TeacherID").Range).Value = TeacherID

Intersect(.Cells, lo.ListColumns("Room").Range).Value = RoomNo

End With

```
modLogs.WriteLog "INSERT", "modSchedule", "New schedule slot created for " & ClassLvl & " (" & Subj  
& ")"
```

```
AddScheduleSlot = True
```

```
Exit Function
```

```
ErrorHandler:
```

```
AddScheduleSlot = False
```

```
MsgBox Err.Description, vbCritical, "Scheduling Error"
```

```
End Function
```

```
Private Function HasConflict(DayName As String, StartT As String, T_ID As Long) As Boolean
```

```
 ' -----
```

```
 ' PURPOSE: Prevent double-booking teachers.
```

```
 ' -----
```

```
Dim lo As ListObject
```

```
Dim r As Range
```

```
Set lo = ThisWorkbook.Sheets("Schedule").ListObjects("tblSchedule")
```

```
If lo.ListRows.count > 0 Then
```

```
    For Each r In lo.ListColumns("DayOfWeek").DataBodyRange
```

```
        If r.Value = DayName Then
```

```
            ' Check if same teacher and same start time
```

```
            If Intersect(r.EntireRow, lo.ListColumns("TeacherID").Range).Value = T_ID And _
```

```
                Intersect(r.EntireRow, lo.ListColumns("StartTime").Range).Value = StartT Then
```



```
        HasConflict = True

        Exit Function

    End If

End If

Next r

End If

HasConflict = False

End Function
```

Option Explicit

```
'
=====
===

' Module: modSecurity

' Purpose: Handles authentication, password hashing, and user session management.

' Author: Senior Enterprise Architect

'
=====
===
```

Public Function HashPassword(ByVal PlainText As String) As String

```
    ' -----

    ' PURPOSE: Obfuscates a string into a Hex-shifted format to protect passwords.

    ' -----

    Dim i As Integer
```

```
Dim strHash As String
```

```
Dim charCode As Integer
```

```
On Error GoTo ErrorHandler
```

```
If Len(PlainText) = 0 Then
```

```
    HashPassword = ""
```

```
    Exit Function
```

```
End If
```

```
' Simple enterprise VBA obfuscation (Shift ASCII + 15, then convert to Hex)
```

```
For i = 1 To Len(PlainText)
```

```
    charCode = Asc(Mid(PlainText, i, 1)) + 15
```

```
    strHash = strHash & Hex(charCode)
```

```
Next i
```

```
HashPassword = strHash
```

```
Exit Function
```

```
ErrorHandler:
```

```
    MsgBox "Security Error: Hash generation failed.", vbCritical, "modSecurity.HashPassword"
```

```
    HashPassword = ""
```

```
End Function
```

```
Public Sub CreateDefaultAdmin()
```

```

' -----
' PURPOSE: Creates a default system administrator if the 'admin' account is missing.
'         Handles Excel's default blank row behavior gracefully.
' -----

Dim lo As ListObject

Dim lr As ListRow

Dim adminRow As Long

On Error GoTo ErrorHandler

' 1. Check if the admin account already exists

adminRow = modDatabase.FindRecordRow("tblUsers", "Username", "admin")

' 2. If adminRow is 0, the admin does not exist. We must create it.

If adminRow = 0 Then

    Set lo = GetTable("tblUsers")

    If lo Is Nothing Then Exit Sub

    ' Handle Excel's default "Ghost Row" (a single blank row)

    If lo.ListRows.count = 1 Then

        If Trim(lo.DataBodyRange.Cells(1, 1).Value) = "" Then

            ' The row is blank, let's reuse it instead of creating a new one

            Set lr = lo.ListRows(1)

        Else

            ' The row has data (but isn't admin), add a new one

```

```

        Set lr = lo.ListRows.Add

    End If

Else

    ' Add a completely new row

    Set lr = lo.ListRows.Add

End If


' 3. Inject Admin Credentials

Intersect(lr.Range, lo.ListColumns("UserID").Range).Value = 1

Intersect(lr.Range, lo.ListColumns("Username").Range).Value = "admin"

Intersect(lr.Range, lo.ListColumns("PasswordHash").Range).Value = HashPassword("admin123")

Intersect(lr.Range, lo.ListColumns("Role").Range).Value = UserRole.Admin ' Enum from modGlobal

Intersect(lr.Range, lo.ListColumns("LastLogin").Range).Value = Now

Intersect(lr.Range, lo.ListColumns("Status").Range).Value = RecordStatus.Active ' Enum from
modGlobal


    Debug.Print "System Bootstrapped: Default Admin created (admin / admin123)"

End If


Exit Sub


ErrorHandler:

MsgBox "Security Error: Failed to generate default admin." & vbCrLf & _

    "Details: " & Err.Description, vbCritical, "modSecurity.CreateDefaultAdmin"

End Sub

```

```
Public Function AuthenticateUser(ByVal strUsername As String, ByVal strPassword As String) As Boolean
```

```
    ' -----
```

```
    ' PURPOSE: Validates credentials against tblUsers and initializes the session.
```

```
    ' -----
```

```
    Dim lo As ListObject
```

```
    Dim userRowIdx As Long
```

```
    Dim storedHash As String
```

```
    Dim accountStatus As Integer
```

```
    On Error GoTo ErrorHandler
```

```
    AuthenticateUser = False ' Default to fail
```

```
    ' 1. Find the user in the database
```

```
    userRowIdx = FindRecordRow("tblUsers", "Username", strUsername)
```

```
    ' If row is 0, user does not exist
```

```
    If userRowIdx = 0 Then Exit Function
```

```
    ' 2. Retrieve user data
```

```
    Set lo = GetTable("tblUsers")
```

```
    ' Extract data using Intersect for robust column referencing
```

```
    storedHash = Intersect(lo.ListRows(userRowIdx).Range, lo.ListColumns("PasswordHash").Range).Value
```

```
    accountStatus = Intersect(lo.ListRows(userRowIdx).Range, lo.ListColumns("Status").Range).Value
```

' 3. Security Checks

If accountStatus <> RecordStatus.Active Then

 MsgBox "Authentication Failed: Account is disabled or deleted.", vbExclamation, "Access Denied"

 Exit Function

End If

If storedHash <> HashPassword(strPassword) Then

 Exit Function ' Silent exit on bad password, Form will handle UI warning

End If

' 4. Initialize Global Session State

g_CurrentUserID = Intersect(lo.ListRows(userRowIdx).Range, lo.ListColumns("UserID").Range).Value

g_CurrentUsername = strUsername

g_CurrentUserRole = Intersect(lo.ListRows(userRowIdx).Range, lo.ListColumns("Role").Range).Value

g_IsLoggedIn = True

' Update Last Login Time

Intersect(lo.ListRows(userRowIdx).Range, lo.ListColumns("LastLogin").Range).Value = Now

AuthenticateUser = True

Exit Function

ErrorHandler:

MsgBox "Security Error: Authentication process failed." & vbCrLf & _

```
"Details: " & Err.Description, vbCritical, "modSecurity.AuthenticateUser"
```

```
End Function
```

```
Public Sub LogoutUser()
```

```
' -----
```

```
' PURPOSE: Safely clears the session state.
```

```
' -----
```

```
g_CurrentUserID = 0
```

```
g_CurrentUsername = ""
```

```
g_CurrentUserRole = 0
```

```
g_IsLoggedIn = False
```

```
End Sub
```

```
Option Explicit
```

```
'
```

```
=====
```

```
===
```

```
' Module: modStudents
```

```
' Purpose: Business logic and Data Access for Student records.
```

```
' Author: Senior Enterprise Architect
```

```
'
```

```
=====
```

```
===
```

```
Public Function RegisterNewStudent(ByVal FirstName As String, ByVal LastName As String, _
```

```
ByVal Gender As String, ByVal DOB As Date, _
```

```

        ByVal ClassLevel As String, ByVal Section As String, _
        ByVal ParentName As String, ByVal Phone As String) As Boolean

' -----

' PURPOSE: Safely writes a new student record to tblStudents with an auto-generated ID.
' RETURNS: True if successful, False if failed.
' -----

Dim lo As ListObject
Dim lr As ListRow
Dim newID As Long

On Error GoTo ErrorHandler

RegisterNewStudent = False ' Default return value

' 1. Validate critical inputs (Basic backend protection)
If Trim(FirstName) = "" Or Trim(LastName) = "" Or Trim(ClassLevel) = "" Then
    MsgBox "Validation Error: First Name, Last Name, and Class are strictly required.", vbExclamation,
    "Data Rejected"

    Exit Function
End If

' 2. Connect to the database table
Set lo = modDatabase.GetTable("tblStudents")

If lo Is Nothing Then Err.Raise vbObjectError + 1, , "tblStudents not found."

' 3. Generate a new sequential Student ID
newID = modDatabase.GetNextID("tblStudents", "StudentID")

```



```
If newID <= 0 Then Err.Raise vbObjectError + 2, , "Failed to generate Student ID."
```

```
' 4. Add the new record
```

```
Set lr = lo.ListRows.Add
```

```
' 5. Map the data to the correct columns securely
```

```
Intersect(lr.Range, lo.ListColumns("StudentID").Range).Value = newID
```

```
Intersect(lr.Range, lo.ListColumns("FirstName").Range).Value = Trim(FirstName)
```

```
Intersect(lr.Range, lo.ListColumns("LastName").Range).Value = Trim(LastName)
```

```
Intersect(lr.Range, lo.ListColumns("Gender").Range).Value = Trim(Gender)
```

```
Intersect(lr.Range, lo.ListColumns("DOB").Range).Value = DOB
```

```
Intersect(lr.Range, lo.ListColumns("Class").Range).Value = Trim(ClassLevel)
```

```
Intersect(lr.Range, lo.ListColumns("Section").Range).Value = Trim(Section)
```

```
Intersect(lr.Range, lo.ListColumns("ParentName").Range).Value = Trim(ParentName)
```

```
Intersect(lr.Range, lo.ListColumns("Phone").Range).Value = Trim(Phone)
```

```
Intersect(lr.Range, lo.ListColumns("RegistrationDate").Range).Value = Date ' Today's date
```

```
Intersect(lr.Range, lo.ListColumns("Status").Range).Value = RecordStatus.Active ' Enum from  
modGlobal
```

```
' 6. Optional: Log the activity (We will build full logging in a later phase)
```

```
Debug.Print "Student Registered: " & FirstName & " " & LastName & " (ID: " & newID & ")"
```

```
RegisterNewStudent = True
```

```
Exit Function
```

ErrorHandler:

```
MsgBox "Database Error: Could not register student." & vbCrLf & _
```

```
"Details: " & Err.Description, vbCritical, "modStudents.RegisterNewStudent"
```

```
End Function
```

```
Option Explicit
```

```
'
```

```
=====
```

```
' Module: modSystemSetup
```

```
' Purpose: Bootstraps the enterprise architecture by creating all required worksheets
```

```
' and foundational database tables (ListObjects).
```

```
' Author: Senior Enterprise Architect
```

```
'
```

```
=====
```

```
Public Sub InitializeSystemArchitecture()
```

```
' -----
```

```
' PURPOSE: Creates sheets and tables if they do not exist.
```

```
' -----
```

```
On Error GoTo ErrorHandler
```

```
' Optimize performance during setup
```

```
Application.ScreenUpdating = False
```

```
Application.Calculation = xlCalculationManual
```

```
Application.DisplayAlerts = False
```

```
Dim arrSheets As Variant
```

```
Dim i As Integer
```

```
Dim ws As Worksheet
```

```
' Define all required system worksheets
```

```
arrSheets = Array("Dashboard", "Students", "Results", "Attendance", _  
                  "Teachers", "Schedule", "Users", "Settings", _  
                  "Subjects", "Logs", "Temp", "ReportCard")
```

```
' 1. Create Worksheets
```

```
For i = LBound(arrSheets) To UBound(arrSheets)
```

```
    If Not SheetExists(CStr(arrSheets(i))) Then
```

```
        Set ws = ThisWorkbook.Sheets.Add(After:=ThisWorkbook.Sheets(ThisWorkbook.Sheets.count))
```

```
        ws.Name = arrSheets(i)
```

```
    End If
```

```
Next i
```

```
' 2. Build Core Database Tables
```

```
Call BuildTable("Students", "tblStudents",  
"StudentID,FirstName,LastName,Gender,DOB,Class,Section,ParentName,Phone,RegistrationDate,Status"  
)
```

```
Call BuildTable("Users", "tblUsers", "UserID,Username>PasswordHash,Role,LastLogin,Status")
```

```
Call BuildTable("Logs", "tblLogs", "LogID,Timestamp,UserID,Action,Module,Details")
```

```
Call BuildTable("Settings", "tblSettings", "SettingKey,SettingValue,Description")
```

' 3. Cleanup & Optimization Restoration

Application.ScreenUpdating = True

Application.Calculation = xlCalculationAutomatic

Application.DisplayAlerts = True

MsgBox "System Architecture Initialized Successfully!", vbInformation, "Setup Complete"

Exit Sub

ErrorHandler:

' Enterprise standard error handling

Application.ScreenUpdating = True

Application.Calculation = xlCalculationAutomatic

Application.DisplayAlerts = True

MsgBox "Error " & Err.Number & ": " & Err.Description & vbCrLf & _

 "Module: modSystemSetup" & vbCrLf & _

 "Procedure: InitializeSystemArchitecture", vbCritical, "Architecture Setup Error"

End Sub

Private Function SheetExists(SheetName As String) As Boolean

' -----

' PURPOSE: Helper function to safely check if a worksheet exists.

' -----

Dim ws As Worksheet

On Error Resume Next

```

Set ws = ThisWorkbook.Sheets(SheetName)

On Error GoTo 0

SheetExists = Not (ws Is Nothing)

End Function


Private Sub BuildTable(SheetName As String, TableName As String, ColumnHeaders As String)

' -----

' PURPOSE: Creates a ListObject (Table) with predefined columns safely.

' -----

Dim ws As Worksheet

Dim lo As ListObject

Dim arrHeaders As Variant

Dim i As Integer


On Error GoTo ErrorHandler


Set ws = ThisWorkbook.Sheets(SheetName)


' Check if table already exists to prevent duplicate errors

On Error Resume Next

Set lo = ws.ListObjects(TableName)

On Error GoTo ErrorHandler


If lo Is Nothing Then

    arrHeaders = Split(ColumnHeaders, ",")

```

```
' Write headers to the first row
```

```
For i = LBound(arrHeaders) To UBound(arrHeaders)
```

```
    ws.Cells(1, i + 1).Value = arrHeaders(i)
```

```
Next i
```

```
' Create the ListObject
```

```
Set lo = ws.ListObjects.Add(xlSrcRange, ws.Range(ws.Cells(1, 1), ws.Cells(2, UBound(arrHeaders) + 1)), , xlYes)
```

```
lo.Name = TableName
```

```
lo.TableStyle = "TableStyleMedium2" ' Professional styling
```

```
End If
```

```
Exit Sub
```

```
ErrorHandler:
```

```
Err.Raise Err.Number, "BuildTable", Err.Description
```

```
End Sub
```

```
Option Explicit
```

```
'
```

```
=====
```

```
' Module: modTeachers
```

```
' Purpose: Business logic and Data Access for Teacher Management.
```

' Author: Senior Enterprise Architect

'

=====

Public Sub SetupTeachersTable()

' -----

' PURPOSE: Bootstraps the tblTeachers database structure safely.

' -----

Dim ws As Worksheet

Dim lo As ListObject

Dim headers As String

On Error GoTo ErrorHandler

Set ws = ThisWorkbook.Sheets("Teachers")

headers = "TeacherID,FullName,Gender,Phone,Specialization,AssignedClass,JoinDate,Status"

' Check if table exists

On Error Resume Next

Set lo = ws.ListObjects("tblTeachers")

On Error GoTo ErrorHandler

If lo Is Nothing Then

' Utilize our modSystemSetup logic style for consistency

Dim arrHeaders As Variant, i As Integer

arrHeaders = Split(headers, ",")

```
For i = LBound(arrHeaders) To UBound(arrHeaders)
```

```
    ws.Cells(1, i + 1).Value = arrHeaders(i)
```

```
Next i
```

```
Set lo = ws.ListObjects.Add(xlSrcRange, ws.Range(ws.Cells(1, 1), ws.Cells(2, UBound(arrHeaders) + 1)), , xlYes)
```

```
lo.Name = "tblTeachers"
```

```
lo.TableStyle = "TableStyleMedium5" ' Distinct purple/grey style for Teachers
```

```
modLogs.WriteLog "INIT", "modTeachers", "Teacher table initialized."
```

```
End If
```

```
Exit Sub
```

```
ErrorHandler:
```

```
MsgBox "Critical Error in SetupTeachersTable: " & Err.Description, vbCritical
```

```
End Sub
```

```
Public Function RegisterTeacher( _
```

```
    ByVal FullName As String, _
```

```
    ByVal Gender As String, _
```

```
    ByVal Phone As String, _
```

```
    ByVal Specialization As String, _
```

```
    ByVal AssignedClass As String) As Boolean
```

```
' -----
```

```
' PURPOSE: Adds a new teacher to the system with full validation and logging.
```



```

' -----

Dim lo As ListObject

Dim lr As ListRow

Dim newID As Long

On Error GoTo ErrorHandler

' 1. Basic Validation

If Trim(FullName) = "" Or Trim(Phone) = "" Then

    Err.Raise vbObjectError + 513, "modTeachers", "Teacher name and phone are mandatory."

End If

' 2. Get Database Reference

Set lo = ThisWorkbook.Sheets("Teachers").ListObjects("tblTeachers")

' 3. Generate New ID (Assuming modDatabase.GetNextID exists from your PDF)

newID = modDatabase.GetNextID("tblTeachers", "TeacherID")

' 4. Add Record

Set lr = lo.ListRows.Add

With lr.Range

    Intersect(lr.Range, lo.ListColumns("TeacherID").Range).Value = newID

    Intersect(lr.Range, lo.ListColumns("FullName").Range).Value = StrConv(Trim(FullName),
vbProperCase)

    Intersect(lr.Range, lo.ListColumns("Gender").Range).Value = Gender

    Intersect(lr.Range, lo.ListColumns("Phone").Range).Value = Phone

```

```
Intersect(lr.Range, lo.ListColumns("Specialization").Range).Value = Specialization
```

```
Intersect(lr.Range, lo.ListColumns("AssignedClass").Range).Value = AssignedClass
```

```
Intersect(lr.Range, lo.ListColumns("JoinDate").Range).Value = Date
```

```
Intersect(lr.Range, lo.ListColumns("Status").Range).Value = "Active"
```

```
End With
```

```
' 5. Audit Trail Integration
```

```
modLogs.WriteLog "INSERT", "modTeachers", "New teacher registered: " & FullName & " (ID: " &  
newID & ")"
```

```
RegisterTeacher = True
```

```
Exit Function
```

```
ErrorHandler:
```

```
RegisterTeacher = False
```

```
MsgBox "Error Registering Teacher: " & Err.Description, vbCritical, "Database Error"
```

```
End Function
```

```
Option Explicit
```

```
,
```

```
=====
```

```
===
```

```
' Module: modUtilities
```

```
' Purpose: Core utility functions used across the entire application.
```

```
' Author: Senior Enterprise Architect
```

```
'  
=====
```

```
Public Sub OptimizeApp(ByVal TurnOnSpeed As Boolean)
```

```
' -----
```

```
' PURPOSE: Toggles Excel application settings to maximize VBA execution speed.
```

```
' PARAMETERS: TurnOnSpeed - True turns off screen updates; False restores normal UI.
```

```
' -----
```

```
On Error GoTo ErrorHandler
```

```
With Application
```

```
    If TurnOnSpeed Then
```

```
        .ScreenUpdating = False
```

```
        .Calculation = xlCalculationManual
```

```
        .EnableEvents = False
```

```
        ' Optional: .DisplayStatusBar = False (Kept true if we want progress updates)
```

```
    Else
```

```
        .ScreenUpdating = True
```

```
        .Calculation = xlCalculationAutomatic
```

```
        .EnableEvents = True
```

```
    End If
```

```
End With
```

```
Exit Sub
```

```
ErrorHandler:
```

```

' If an error occurs, force Excel back to normal so the user isn't frozen

Application.ScreenUpdating = True

Application.Calculation = xlCalculationAutomatic

Application.EnableEvents = True

MsgBox "Critical Optimization Error: " & Err.Description, vbCritical, "modUtilities"

End Sub

```

UserForms

Option Explicit

```

'
=====
===

```

' UserForm: frmAttendance

' Purpose: Frontend UI for rapid, bulk attendance tracking.

' Author: Senior Enterprise Architect

```

'
=====
===

```

Private Sub UserForm_Initialize()

```

' -----
' PURPOSE: Sets up default values when the form loads.
' -----

```

On Error GoTo ErrorHandler

' Default to today's date

Me.txtDate.Text = Format(Date, "mm/dd/yyyy")

' Populate Classes

With Me.cmbClass

.Clear

.AddItem "Grade 1"

.AddItem "Grade 2"

.AddItem "Grade 3"

.AddItem "Grade 4"

.AddItem "Grade 5"

.AddItem "Grade 6"

.AddItem "Grade 7"

.AddItem "Grade 8"

End With

' Populate Sections

With Me.cmbSection

.Clear

.AddItem "A"

.AddItem "B"

.AddItem "C"

.AddItem "D"

End With

Exit Sub

ErrorHandler:

MsgBox "UI Load Error: " & Err.Description, vbCritical

End Sub

Private Sub cmdLoad_Click()

' -----

' PURPOSE: Queries tblStudents and loads matching students into the ListBox.

' -----

Dim loStudents As ListObject

Dim lr As ListRow

Dim classVal As String

Dim sectionVal As String

Dim studentName As String

Dim studentID As Long

On Error GoTo ErrorHandler

' 1. Validate Form

If Me.cmbClass.ListIndex = -1 Or Me.cmbSection.ListIndex = -1 Then

MsgBox "Please select both Class and Section to load the roster.", vbExclamation, "Validation"

Exit Sub

End If

classVal = Me.cmbClass.Value

```
sectionVal = Me.cmbSection.Value
```

```
Me.lstStudents.Clear
```

```
' 2. Get Database Table
```

```
Set loStudents = modDatabase.GetTable("tblStudents")
```

```
If loStudents Is Nothing Then Exit Sub
```

```
' 3. Loop through students and populate roster
```

```
For Each lr In loStudents.ListRows
```

```
    ' Only load if Class, Section match, and student is Active (Status = 1)
```

```
    If Intersect(lr.Range, loStudents.ListColumns("Class").Range).Value = classVal And _
```

```
        Intersect(lr.Range, loStudents.ListColumns("Section").Range).Value = sectionVal And _
```

```
        Intersect(lr.Range, loStudents.ListColumns("Status").Range).Value = RecordStatus.Active Then
```

```
        studentID = Intersect(lr.Range, loStudents.ListColumns("StudentID").Range).Value
```

```
        studentName = Intersect(lr.Range, loStudents.ListColumns("FirstName").Range).Value & " " & _
```

```
            Intersect(lr.Range, loStudents.ListColumns("LastName").Range).Value
```

```
        ' Add to ListBox (Column 0 = ID, Column 1 = Name)
```

```
        Me.lstStudents.AddItem studentID
```

```
        Me.lstStudents.List(Me.lstStudents.ListCount - 1, 1) = studentName
```

```
    End If
```

```
Next lr
```

```
If Me.lstStudents.ListCount = 0 Then
```

```
MsgBox "No active students found for this Class and Section.", vbInformation, "Empty Roster"
```

```
End If
```

```
Exit Sub
```

```
ErrorHandler:
```

```
MsgBox "Error loading roster: " & Err.Description, vbCritical
```

```
End Sub
```

```
Private Sub cmdPresent_Click()
```

```
Call ProcessAttendance("Present")
```

```
End Sub
```

```
Private Sub cmdAbsent_Click()
```

```
Call ProcessAttendance("Absent")
```

```
End Sub
```

```
Private Sub ProcessAttendance(ByVal StatusType As String)
```

```
' -----
```

```
' PURPOSE: Loops through selected students in the ListBox and saves their attendance.
```

```
' -----
```

```
Dim i As Integer
```

```
Dim studentID As Long
```

```
Dim processCount As Integer
```

```
Dim classDate As Date
```

```
Dim success As Boolean
```


On Error GoTo ErrorHandler

' 1. Validation

If Not IsDate(Me.txtDate.Text) Then

 MsgBox "Please enter a valid date.", vbExclamation, "Validation"

 Exit Sub

End If

classDate = CDate(Me.txtDate.Text)

If Me.lstStudents.ListIndex = -1 Then

 MsgBox "Please select at least one student from the roster.", vbExclamation, "Validation"

 Exit Sub

End If

' 2. Loop through ListBox and save

processCount = 0

For i = 0 To Me.lstStudents.ListCount - 1

 If Me.lstStudents.Selected(i) = True Then

 ' Column 0 holds the hidden StudentID

 studentID = CLng(Me.lstStudents.List(i, 0))

 ' Fire to backend engine

 success = modAttendance.RecordAttendance(classDate, studentID, StatusType)

If success Then

processCount = processCount + 1

' Deselect them visually so the teacher knows they are processed

Me.lstStudents.Selected(i) = False

End If

End If

Next i

MsgBox processCount & " students successfully marked as " & StatusType & ".", vbInformation,
"Success"

Exit Sub

ErrorHandler:

MsgBox "Error processing attendance: " & Err.Description, vbCritical

End Sub

Private Sub cmdClose_Click()

Unload Me

End Sub

Option Explicit

'

=====

===

' UserForm: frmDashboard

' Purpose: Central navigation switchboard for the application.

' Author: Senior Enterprise Architect

,

=====
===

Private Sub UserForm_Initialize()

' -----

' PURPOSE: Configures the dashboard based on the logged-in user's state.

' -----

Dim roleName As String

On Error GoTo ErrorHandler

' 1. Set personalized greeting

Me.lblWelcome.Caption = "Welcome, " & g_CurrentUsername & "!"

' 2. Translate Role Integer to readable string

Select Case g_CurrentUserRole

Case UserRole.Admin

 roleName = "Administrator"

Case UserRole.Teacher

 roleName = "Teacher"

Case UserRole.Viewer

 roleName = "Viewer"

Case Else

```
        roleName = "Unknown Access"
```

```
End Select
```

```
Me.lblRole.Caption = "Access Level: " & roleName
```

```
' 3. Set Date (We will upgrade this to Ethiopian Date in Phase 7)
```

```
Me.lblDate.Caption = Format(Now, "dddd, mmmm dd, yyyy")
```

```
' 4. Security: Disable certain buttons based on role
```

```
If g_CurrentUserRole = UserRole.Viewer Then
```

```
    Me.cmdStudents.Enabled = False
```

```
    Me.cmdTeachers.Enabled = False
```

```
End If
```

```
Exit Sub
```

```
ErrorHandler:
```

```
    MsgBox "Error loading Dashboard: " & Err.Description, vbCritical, "System Error"
```

```
End Sub
```

```
' --- NAVIGATION EVENTS ---
```

```
Private Sub cmdStudents_Click()
```

```
    ' Launch the Student Management module
```

```
    frmStudents.Show vbModal
```

```
End Sub
```

```
Private Sub cmdTeachers_Click()
```

```
    frmTeachers.Show vbModal
```

```
End Sub
```

```
Private Sub cmdAttendance_Click()
```

```
    ' Launch the Attendance module
```

```
    frmAttendance.Show vbModal
```

```
End Sub
```

```
Private Sub cmdResults_Click()
```

```
    ' Launch the Results module
```

```
    frmResults.Show vbModal
```

```
End Sub
```

```
Private Sub cmdLogout_Click()
```

```
    ' -----
```

```
    ' PURPOSE: Safely closes the dashboard, triggering the application shutdown sequence.
```

```
    ' -----
```

```
    If MsgBox("Are you sure you want to log out?", vbYesNo + vbQuestion, "Confirm Logout") = vbYes  
Then
```

```
    Unload Me
```

```
End If
```

```
End Sub
```

```
Private Sub UserForm_QueryClose(Cancel As Integer, CloseMode As Integer)
```

```

' -----
' PURPOSE: Prevents bypassing logout by clicking the red 'X'
' -----

' If the user clicks the X (CloseMode = 0), force them to use the Logout button
' Note: We allow this if they are legitimately closing the app, but for strict
' enterprise systems, enforcing button clicks prevents accidental data loss.

If CloseMode = 0 Then

    Cancel = True

    MsgBox "Please use the 'Secure Logout' button to exit.", vbExclamation, "Action Blocked"

End If

End Sub

```

Option Explicit

```

Private Sub lblError_Click()

```

```

End Sub

```

```

'

```

```

=====
===

```

```

' UserForm: frmLogin

```

```

' Purpose: Captures user credentials and passes them to the Security Engine.

```

```

' Author: Senior Enterprise Architect

```

```

'

```

```

=====
===

```

```
Private Sub UserForm_Initialize()
```

```
' -----
```

```
' PURPOSE: Runs immediately when the form is loaded into memory.
```

```
' -----
```

```
On Error GoTo ErrorHandler
```

```
' Clear fields and hide error messages
```

```
Me.txtUsername.Text = ""
```

```
Me.txtPassword.Text = ""
```

```
Me.lblError.Caption = ""
```

```
' Ensure the admin account exists in case this is the very first run
```

```
modSecurity.CreateDefaultAdmin
```

```
Exit Sub
```

```
ErrorHandler:
```

```
MsgBox "Error initializing login form: " & Err.Description, vbCritical, "System Error"
```

```
End Sub
```

```
Private Sub cmdLogin_Click()
```

```
' -----
```

```
' PURPOSE: Triggers when the user clicks Login or presses the Enter key.
```

```
' -----
```

```
Dim strUser As String
```

```
Dim strPass As String
```

```
Dim isAuthenticated As Boolean
```

```
On Error GoTo ErrorHandler
```

```
' 1. Form Validation (Don't waste DB resources on empty inputs)
```

```
strUser = Trim(Me.txtUsername.Text)
```

```
strPass = Trim(Me.txtPassword.Text)
```

```
If strUser = "" Or strPass = "" Then
```

```
    ShowError "Please enter both Username and Password."
```

```
    Exit Sub
```

```
End If
```

```
' Visual feedback for the user
```

```
Me.lblError.ForeColor = &H80000008& ' Standard Black
```

```
Me.lblError.Caption = "Authenticating..."
```

```
DoEvents ' Forces UI to update immediately
```

```
' 2. Pass to Security Engine
```

```
isAuthenticated = modSecurity.AuthenticateUser(strUser, strPass)
```

```
' 3. Handle Result
```

```
If isAuthenticated Then
```

```
    ' Success!
```



```
Me.lblError.ForeColor = &HC000& ' Green
```

```
Me.lblError.Caption = "Success! Loading..."
```

```
DoEvents
```

```
' In the future, we will launch frmDashboard here.
```

```
MsgBox "Welcome, " & g_CurrentUsername & "!", vbInformation, "Access Granted"
```

```
' Unload the form from memory
```

```
Unload Me
```

```
Else
```

```
' Failure
```

```
ShowError "Invalid username or password."
```

```
' Security best practice: Clear the password field on failure
```

```
Me.txtPassword.Text = ""
```

```
Me.txtPassword.SetFocus
```

```
End If
```

```
Exit Sub
```

```
ErrorHandler:
```

```
ShowError "System error during authentication."
```

```
Debug.Print "Login Error: " & Err.Description
```

```
End Sub
```

```
Private Sub cmdCancel_Click()
```

```
 ' -----
```

```
 ' PURPOSE: Safely closes the application if the user cancels login.
```

```
 ' -----
```

```
Unload Me
```

```
 ' Note: In a fully locked down system, we would force the workbook to close here:
```

```
 ' ThisWorkbook.Close SaveChanges:=False
```

```
End Sub
```

```
Private Sub ShowError(ByVal ErrorMessage As String)
```

```
 ' -----
```

```
 ' PURPOSE: Centralized UI error handling for the form.
```

```
 ' -----
```

```
Me.lblError.ForeColor = &HFF& ' Red
```

```
Me.lblError.Caption = ErrorMessage
```

```
End Sub
```

```
Option Explicit
```

```
Private Sub lstStudents_Click()
```

```
End Sub
```

```

'
=====
===

' UserForm: frmReportCard

' Purpose: UI for filtering students and batch-generating report cards.
'
=====
===

Private Sub UserForm_Initialize()

' Initialize dropdowns with unique values from the database

LoadFilters

Me.cmbTerm.List = Array("Term 1", "Term 2", "Term 3", "Final Exam")

End Sub


Private Sub LoadFilters()

' Fetches unique classes and sections from tblStudents

Dim lo As ListObject

Dim cell As Range

Dim dictClass As Object, dictSection As Object

Set dictClass = CreateObject("Scripting.Dictionary")

Set dictSection = CreateObject("Scripting.Dictionary")


On Error Resume Next

Set lo = modDatabase.GetTable("tblStudents")

' Extract unique Class values

```

```
For Each cell In lo.ListColumns("Class").DataBodyRange
```

```
    If Not dictClass.Exists(cell.Value) Then dictClass.Add cell.Value, cell.Value
```

```
Next cell
```

```
' Extract unique Section values
```

```
For Each cell In lo.ListColumns("Section").DataBodyRange
```

```
    If Not dictSection.Exists(cell.Value) Then dictSection.Add cell.Value, cell.Value
```

```
Next cell
```

```
Me.cmbClass.List = dictClass.Items
```

```
Me.cmbSection.List = dictSection.Items
```

```
On Error GoTo 0
```

```
End Sub
```

```
Private Sub cmbClass_Change()
```

```
    UpdateStudentList
```

```
End Sub
```

```
Private Sub cmbSection_Change()
```

```
    UpdateStudentList
```

```
End Sub
```

```
Private Sub UpdateStudentList()
```

' Filters the ListBox based on ComboBox selections

Dim lo As ListObject

Dim lr As ListRow

Dim matchClass As Boolean, matchSection As Boolean

Set lo = modDatabase.GetTable("tblStudents")

Me.lstStudents.Clear

If lo.ListRows.count = 0 Then Exit Sub

For Each lr In lo.ListRows

 matchClass = (Me.cmbClass.Value = "" Or Intersect(lr.Range, lo.ListColumns("Class").Range).Value = Me.cmbClass.Value)

 matchSection = (Me.cmbSection.Value = "" Or Intersect(lr.Range, lo.ListColumns("Section").Range).Value = Me.cmbSection.Value)

If matchClass And matchSection Then

 With Me.lstStudents

 .AddItem Intersect(lr.Range, lo.ListColumns("FirstName").Range).Value & " " & _

 Intersect(lr.Range, lo.ListColumns("LastName").Range).Value

 .List(.ListCount - 1, 1) = Intersect(lr.Range, lo.ListColumns("StudentID").Range).Value

 End With

End If

Next lr

End Sub

Private Sub cmdGenerate_Click()

' Batch generation logic

Dim i As Integer

Dim selectedID As Long

Dim term As String

Dim count As Integer

term = Me.cmbTerm.Value

If term = "" Then

MsgBox "Please select an Exam Term first.", vbExclamation

Exit Sub

End If

count = 0

For i = 0 To Me.lstStudents.ListCount - 1

If Me.lstStudents.Selected(i) Then

selectedID = CLng(Me.lstStudents.List(i, 1))

' Call our backend module function

If modReportCard.GenerateReportCard(selectedID, term) Then

' In a real enterprise app, we would export to PDF here

```

        ' For now, we just notify

        count = count + 1

    End If

End If

Next i


If count > 0 Then

    MsgBox count & " Report Cards generated successfully on the 'ReportCard' sheet.", vbInformation

Else

    MsgBox "No students selected.", vbExclamation

End If


End Sub


Private Sub cmdPreview_Click()

' Show the worksheet to the user

ThisWorkbook.Sheets("ReportCard").Visible = xlSheetVisible

ThisWorkbook.Sheets("ReportCard").Select

Unload Me

End Sub


Private Sub cmdClose_Click()

Unload Me

End Sub

```

Option Explicit

Private Sub lblExam_Click()

End Sub

'

=====

===

' UserForm: frmResults

' Purpose: Frontend UI for validating and submitting exam scores.

' Author: Senior Enterprise Architect

'

=====

===

Private Sub UserForm_Initialize()

' -----

' PURPOSE: Sets up default dropdowns.

' -----

On Error GoTo ErrorHandler

' Populate Exam Names

With Me.cmbExamName

.Clear

.AddItem "Term 1"

.AddItem "Term 2"

.AddItem "Final Exam"

.AddItem "Midterm"

End With

' Populate Classes

With Me.cmbClass

.Clear

.AddItem "Grade 1"

.AddItem "Grade 2"

.AddItem "Grade 3"

.AddItem "Grade 4"

.AddItem "Grade 5"

.AddItem "Grade 6"

.AddItem "Grade 7"

.AddItem "Grade 8"

End With

' Populate Sections

With Me.cmbSection

.Clear

.AddItem "A"

.AddItem "B"

.AddItem "C"

```
.AddItem "D"
```

```
End With
```

```
Exit Sub
```

```
ErrorHandler:
```

```
MsgBox "UI Load Error: " & Err.Description, vbCritical
```

```
End Sub
```

```
Private Sub cmdLoad_Click()
```

```
' -----
```

```
' PURPOSE: Loads students into the Combobox based on Class/Section.
```

```
' -----
```

```
Dim loStudents As ListObject
```

```
Dim lr As ListRow
```

```
Dim classVal As String
```

```
Dim sectionVal As String
```

```
Dim studentName As String
```

```
Dim studentID As Long
```

```
On Error GoTo ErrorHandler
```

```
If Me.cmbClass.ListIndex = -1 Or Me.cmbSection.ListIndex = -1 Then
```

```
MsgBox "Please select a Class and Section first.", vbExclamation
```

```
Exit Sub
```

```
End If
```

classVal = Me.cmbClass.Value

sectionVal = Me.cmbSection.Value

Me.cmbStudent.Clear

Set loStudents = modDatabase.GetTable("tblStudents")

If loStudents Is Nothing Then Exit Sub

For Each lr In loStudents.ListRows

If Intersect(lr.Range, loStudents.ListColumns("Class").Range).Value = classVal And _

Intersect(lr.Range, loStudents.ListColumns("Section").Range).Value = sectionVal And _

Intersect(lr.Range, loStudents.ListColumns("Status").Range).Value = RecordStatus.Active Then

studentID = Intersect(lr.Range, loStudents.ListColumns("StudentID").Range).Value

studentName = Intersect(lr.Range, loStudents.ListColumns("FirstName").Range).Value & " " & _

Intersect(lr.Range, loStudents.ListColumns("LastName").Range).Value

Me.cmbStudent.AddItem studentID

Me.cmbStudent.List(Me.cmbStudent.ListCount - 1, 1) = studentName

End If

Next lr

If Me.cmbStudent.ListCount > 0 Then

Me.cmbStudent.ListIndex = 0 ' Auto-select the first student

MsgBox "Students loaded successfully.", vbInformation

Else

 MsgBox "No students found in this class/section.", vbInformation

End If

Exit Sub

ErrorHandler:

 MsgBox "Error loading students: " & Err.Description, vbCritical

End Sub

Private Sub cmdSave_Click()

 ' -----

 ' PURPOSE: Validates scores and passes them to the Results Engine.

 ' -----

 Dim studentID As Long

 Dim success As Boolean

 On Error GoTo ErrorHandler

 ' 1. Basic Form Validation

 If Me.cmbExamName.Value = "" Or Me.cmbStudent.ListIndex = -1 Then

 MsgBox "Please select an Exam Name and a Student.", vbExclamation, "Validation"

 Exit Sub

 End If

 ' 2. Score Validation (Must be numeric and between 0-100)

```

If Not IsValidScore(Me.txtFaith.Text) Or Not IsValidScore(Me.txtGeez.Text) Or _
    Not IsValidScore(Me.txtHistory.Text) Or Not IsValidScore(Me.txtMorality.Text) Or _
    Not IsValidScore(Me.txtLiturgy.Text) Or Not IsValidScore(Me.txtSaints.Text) Then
    MsgBox "All scores must be valid numbers between 0 and 100.", vbExclamation, "Invalid Scores"
    Exit Sub
End If

```

```

' 3. Extract hidden Student ID from column 0

```

```

studentID = CLng(Me.cmbStudent.Column(0))

```

```

' 4. Send to Backend

```

```

success = modResults.SaveStudentResult( _
    ExamName:=Me.cmbExamName.Value, _
    ClassLevel:=Me.cmbClass.Value, _
    studentID:=studentID, _
    Faith:=CDBl(Me.txtFaith.Text), _
    Geez:=CDBl(Me.txtGeez.Text), _
    History:=CDBl(Me.txtHistory.Text), _
    Morality:=CDBl(Me.txtMorality.Text), _
    Liturgy:=CDBl(Me.txtLiturgy.Text), _
    Saints:=CDBl(Me.txtSaints.Text))

```

```

' 5. Handle Result

```

```

If success Then

```

```

    MsgBox "Scores saved successfully! Total, Average, and Grade have been calculated.",
    vbInformation, "Success"

```

ClearScores

End If

Exit Sub

ErrorHandler:

MsgBox "Error saving results: " & Err.Description, vbCritical

End Sub

Private Function IsValidScore(ByVal ScoreText As String) As Boolean

' -----

' PURPOSE: Helper function to validate numerical bounds.

' -----

' Use standard VBA syntax for returning values from functions

If Trim(ScoreText) = "" Then

IsValidScore = False

Exit Function

End If

If Not IsNumeric(ScoreText) Then

IsValidScore = False

Exit Function

End If

Dim val As Double

val = CDBl(ScoreText)

```
If val >= 0 And val <= 100 Then  
    IsValidScore = True  
Else  
    IsValidScore = False  
End If  
End Function
```

```
Private Sub ClearScores()  
    Me.txtFaith.Text = ""  
    Me.txtGeez.Text = ""  
    Me.txtHistory.Text = ""  
    Me.txtMorality.Text = ""  
    Me.txtLiturgy.Text = ""  
    Me.txtSaints.Text = ""  
    Me.cmbStudent.SetFocus  
End Sub
```

```
Private Sub cmdClose_Click()  
    Unload Me  
End Sub
```

Option Explicit

```
Private Sub cmbSubject_Change()
```

```
End Sub
```

```
Private Sub cmdLoadTeachers_Click()
```

```
End Sub
```

```
Private Sub txtEndTime_Change()
```

```
End Sub
```

```
'
```

```
=====
```

```
' UserForm: frmSchedule
```

```
' Purpose: UI for creating and viewing the Weekly Timetable.
```

```
' Author: Senior Enterprise Architect
```

```
'
```

```
=====
```

```
Private Sub UserForm_Initialize()
```

```
' -----
```

```
' PURPOSE: Initialize dropdowns with data from other modules and setup UI state.
```

```
' -----
```

```
On Error Resume Next
```


Dim i As Integer

' 1. Days of the week (Standard Sunday School Days)

With Me.cmbDay

.AddItem "Monday": .AddItem "Tuesday": .AddItem "Wednesday"

.AddItem "Thursday": .AddItem "Friday": .AddItem "Saturday"

.AddItem "Sunday"

End With

' 2. Subjects (Linking to core requirements)

With Me.cmbSubject

.AddItem "Faith": .AddItem "Geez": .AddItem "History"

.AddItem "Morality": .AddItem "Liturgy": .AddItem "Saints"

End With

' 3. Populate Classes (1-10)

For i = 1 To 10

Me.cmbClass.AddItem "Class " & i

Next i

' 4. Configure Teacher ComboBox for multi-column (Name and ID)

With Me.cmbTeacher

.ColumnCount = 2

.BoundColumn = 2

.TextColumn = 1

```
.ColumnWidths = "120;0" ' Hide the ID column from the user
```

```
End With
```

```
' 5. Configure ListBox for display
```

```
With Me.lstSchedule
```

```
.ColumnCount = 7
```

```
.ColumnWidths = "30;60;50;50;60;80;100" ' ID, Day, Start, End, Class, Subject, Room
```

```
End With
```

```
' 6. Load Data
```

```
LoadTeachersIntoCombo
```

```
RefreshScheduleList
```

```
End Sub
```

```
Private Sub LoadTeachersIntoCombo()
```

```
' -----
```

```
' PURPOSE: Pulls active teachers from tblTeachers into the schedule dropdown.
```

```
' -----
```

```
Dim lo As ListObject
```

```
Dim i As Long
```

```
On Error Resume Next
```

```
Set lo = ThisWorkbook.Sheets("Teachers").ListObjects("tblTeachers")
```

```
On Error GoTo 0
```

```
If lo Is Nothing Then Exit Sub
```

```
Me.cmbTeacher.Clear
```

```
If lo.ListRows.count > 0 Then
```

```
    For i = 1 To lo.ListRows.count
```

```
        ' Column 1: FullName, Column 2: TeacherID (Hidden)
```

```
        Me.cmbTeacher.AddItem lo.DataBodyRange(i, 2).Value
```

```
        Me.cmbTeacher.List(Me.cmbTeacher.ListCount - 1, 1) = lo.DataBodyRange(i, 1).Value
```

```
    Next i
```

```
End If
```

```
End Sub
```

```
Private Sub cmdSave_Click()
```

```
    ' -----
```

```
    ' PURPOSE: Validates UI inputs and commits to the scheduling engine.
```

```
    ' -----
```

```
    Dim tID As Long
```

```
    Dim success As Boolean
```

```
On Error GoTo ErrorHandler
```

```
    ' 1. UI Validation
```

```
If Me.cmbDay.ListIndex = -1 Or Me.cmbTeacher.ListIndex = -1 Or _
```

```
    Me.txtStartTime.Text = "" Or Me.txtEndTime.Text = "" Then
```

```
    MsgBox "Please fill in all required fields (Day, Teacher, Times).", vbExclamation, "Validation Error"
```

Exit Sub

End If

' 2. Get the Bound TeacherID

tID = CLng(Me.cmbTeacher.Value)

' 3. Call backend Engine

success = modSchedule.AddScheduleSlot(_

DayName:=Me.cmbDay.Value, _

StartT:=Me.txtStartTime.Text, _

EndT:=Me.txtEndTime.Text, _

ClassLvl:=Me.cmbClass.Value, _

Subj:=Me.cmbSubject.Value, _

TeacherID:=tID, _

RoomNo:=Me.txtRoom.Text)

' 4. Post-Commit Actions

If success Then

MsgBox "Schedule slot created successfully!", vbInformation, "Success"

RefreshScheduleList

ClearForm

End If

Exit Sub

ErrorHandler:

MsgBox "An error occurred: " & Err.Description, vbCritical, "UI Error"

End Sub

Private Sub RefreshScheduleList()

' -----

' PURPOSE: Synchronizes the ListBox with the tblSchedule data body.

' -----

Dim lo As ListObject

Dim i As Long

On Error Resume Next

Set lo = ThisWorkbook.Sheets("Schedule").ListObjects("tblSchedule")

On Error GoTo 0

Me.lstSchedule.Clear

If lo Is Nothing Then Exit Sub

If lo.ListRows.count = 0 Then Exit Sub

For i = 1 To lo.ListRows.count

With Me.lstSchedule

.AddItem lo.DataBodyRange(i, 1).Value ' ID

.List(.ListCount - 1, 1) = lo.DataBodyRange(i, 2).Value ' Day

.List(.ListCount - 1, 2) = lo.DataBodyRange(i, 3).Value ' Start

```

        .List(.ListCount - 1, 3) = lo.DataBodyRange(i, 4).Value ' End
        .List(.ListCount - 1, 4) = lo.DataBodyRange(i, 5).Value ' Class
        .List(.ListCount - 1, 5) = lo.DataBodyRange(i, 6).Value ' Subject
        .List(.ListCount - 1, 6) = lo.DataBodyRange(i, 8).Value ' Room

    End With

Next i

End Sub

```

```

Private Sub ClearForm()

    Me.cmbDay.ListIndex = -1
    Me.cmbTeacher.ListIndex = -1
    Me.cmbClass.ListIndex = -1
    Me.cmbSubject.ListIndex = -1
    Me.txtStartTime.Text = ""
    Me.txtEndTime.Text = ""
    Me.txtRoom.Text = ""

End Sub

```

```

Private Sub cmdClear_Click()

    ClearForm

End Sub

```

```

Private Sub cmdClose_Click()

    Unload Me

End Sub

```

Option Explicit

```
'  
=====
```

===

```
' UserForm: frmStudents  
' Purpose: Frontend UI for registering and managing student records.  
' Author: Senior Enterprise Architect  
'  
=====
```

===

Private Sub UserForm_Initialize()

```
' -----  
' PURPOSE: Populates the dropdown menus when the form opens.  
' -----
```

On Error GoTo ErrorHandler

' Populate Gender

With Me.cmbGender

.Clear

.AddItem "Male"

.AddItem "Female"

End With

' Populate Classes (We will hardcode for now, can be moved to a DB table later)

With Me.cmbClass

.Clear

.AddItem "Grade 1"

.AddItem "Grade 2"

.AddItem "Grade 3"

.AddItem "Grade 4"

.AddItem "Grade 5"

.AddItem "Grade 6"

.AddItem "Grade 7"

.AddItem "Grade 8"

End With

' Populate Sections

With Me.cmbSection

.Clear

.AddItem "A"

.AddItem "B"

.AddItem "C"

.AddItem "D"

End With

Exit Sub

ErrorHandler:


```
MsgBox "Error loading form: " & Err.Description, vbCritical
```

```
End Sub
```

```
Private Sub cmdSave_Click()
```

```
    ' -----
```

```
    ' PURPOSE: Validates form input and sends data to the Student Backend Engine.
```

```
    ' -----
```

```
    Dim success As Boolean
```

```
    Dim dtDOB As Date
```

```
    On Error GoTo ErrorHandler
```

```
    ' 1. Frontend Validation
```

```
    If Trim(Me.txtFirstName.Text) = "" Or Trim(Me.txtLastName.Text) = "" Or _
```

```
        Me.cmbGender.ListIndex = -1 Or Me.cmbClass.ListIndex = -1 Or Trim(Me.txtDOB.Text) = "" Then
```

```
        MsgBox "Please fill in all required fields marked with an asterisk (*).", vbExclamation, "Validation  
Error"
```

```
        Exit Sub
```

```
    End If
```

```
    ' Validate Date of Birth
```

```
    If Not IsDate(Me.txtDOB.Text) Then
```

```
        MsgBox "Please enter a valid Date of Birth.", vbExclamation, "Invalid Date"
```

```
        Me.txtDOB.SetFocus
```

```
        Exit Sub
```

```
    End If
```

```
dtDOB = CDate(Me.txtDOB.Text)
```

```
' 2. Pass data to the Backend Engine
```

```
success = modStudents.RegisterNewStudent( _
```

```
    FirstName:=Me.txtFirstName.Text, _
```

```
    LastName:=Me.txtLastName.Text, _
```

```
    Gender:=Me.cmbGender.Value, _
```

```
    DOB:=dtDOB, _
```

```
    ClassLevel:=Me.cmbClass.Value, _
```

```
    Section:=Me.cmbSection.Value, _
```

```
    ParentName:=Me.txtParentName.Text, _
```

```
    Phone:=Me.txtPhone.Text)
```

```
' 3. Handle Result
```

```
If success Then
```

```
    MsgBox "Student successfully registered!", vbInformation, "Success"
```

```
    ClearForm
```

```
End If
```

```
Exit Sub
```

```
ErrorHandler:
```

```
    MsgBox "An error occurred while saving: " & Err.Description, vbCritical, "UI Error"
```

```
End Sub
```

```
Private Sub cmdClose_Click()
```

```
    Unload Me
```

```
End Sub
```

```
Private Sub ClearForm()
```

```
    ' Helper to reset the form after a successful save
```

```
    Me.txtFirstName.Text = ""
```

```
    Me.txtLastName.Text = ""
```

```
    Me.cmbGender.ListIndex = -1
```

```
    Me.txtDOB.Text = ""
```

```
    Me.cmbClass.ListIndex = -1
```

```
    Me.cmbSection.ListIndex = -1
```

```
    Me.txtParentName.Text = ""
```

```
    Me.txtPhone.Text = ""
```

```
    Me.txtFirstName.SetFocus
```

```
End Sub
```

```
Option Explicit
```

```
,
```

```
=====
```

```
' UserForm: frmTeachers
```

```
' Purpose: UI for managing Sunday School teachers.
```

```
' Author: Senior Enterprise Architect
```

'

=====

Private Sub UserForm_Initialize()

' -----

' PURPOSE: Setup form defaults and load the teacher directory.

' -----

On Error Resume Next

' Populate ComboBoxes

With Me.cmbGender

.AddItem "Male"

.AddItem "Female"

End With

With Me.cmbSpecialization

.AddItem "Faith"

.AddItem "Geez"

.AddItem "History"

.AddItem "Morality"

.AddItem "Liturgy"

.AddItem "Saints"

End With

' Populate Class (Dynamic range or hardcoded for now)

Dim i As Integer

```
For i = 1 To 10
```

```
    Me.cmbClass.AddItem "Class " & i
```

```
Next i
```

```
' Refresh the Directory
```

```
RefreshTeacherList
```

```
End Sub
```

```
Private Sub cmdSave_Click()
```

```
' -----
```

```
' PURPOSE: Validates and sends data to modTeachers.
```

```
' -----
```

```
Dim success As Boolean
```

```
' 1. UI Validation
```

```
If Trim(Me.txtFullName.Text) = "" Or Me.cmbGender.ListIndex = -1 Or Me.txtPhone.Text = "" Then
```

```
    MsgBox "Validation Error: Please fill in Full Name, Gender, and Phone Number.", vbExclamation,  
    "Missing Data"
```

```
    Exit Sub
```

```
End If
```

```
' 2. Call Backend Logic (Enterprise Pattern: UI -> Business Logic)
```

```
success = modTeachers.RegisterTeacher( _
```

```
    FullName:=Me.txtFullName.Text, _
```

```
    Gender:=Me.cmbGender.Value, _
```

```
    Phone:=Me.txtPhone.Text, _
```

```
Specialization:=Me.cmbSpecialization.Value, _  
AssignedClass:=Me.cmbClass.Value)
```

```
' 3. Feedback and Cleanup
```

```
If success Then
```

```
MsgBox "Teacher successfully registered in the system.", vbInformation, "Success"
```

```
RefreshTeacherList
```

```
ClearForm
```

```
End If
```

```
End Sub
```

```
Private Sub RefreshTeacherList()
```

```
' -----
```

```
' PURPOSE: Synchronizes the ListBox with tblTeachers.
```

```
' -----
```

```
Dim lo As ListObject
```

```
Dim dataRange As Range
```

```
Set lo = ThisWorkbook.Sheets("Teachers").ListObjects("tblTeachers")
```

```
' Clear current list
```

```
Me.lstTeachers.Clear
```

```
' Set column properties
```

```
Me.lstTeachers.ColumnCount = 4
```

```
Me.lstTeachers.ColumnWidths = "30;120;60;80" ' ID, Name, Gender, Class
```

```
If lo.ListRows.count > 0 Then
```

```
    Dim i As Long
```

```
    For i = 1 To lo.ListRows.count
```

```
        With Me.lstTeachers
```

```
            .AddItem lo.DataBodyRange(i, 1).Value ' ID
```

```
            .List(.ListCount - 1, 1) = lo.DataBodyRange(i, 2).Value ' Name
```

```
            .List(.ListCount - 1, 2) = lo.DataBodyRange(i, 3).Value ' Gender
```

```
            .List(.ListCount - 1, 3) = lo.DataBodyRange(i, 6).Value ' Assigned Class
```

```
        End With
```

```
    Next i
```

```
End If
```

```
End Sub
```

```
Private Sub ClearForm()
```

```
    Me.txtFullName.Text = ""
```

```
    Me.txtPhone.Text = ""
```

```
    Me.cmbGender.ListIndex = -1
```

```
    Me.cmbSpecialization.ListIndex = -1
```

```
    Me.cmbClass.ListIndex = -1
```

```
    Me.txtFullName.SetFocus
```

```
End Sub
```

```
Private Sub cmdClear_Click()
```

ClearForm

End Sub

Private Sub cmdClose_Click()

Unload Me

End Sub
